

QUESTION 1

$$10011101_2 + 1100110_2 =$$

- A. 253_{10} B. 255_{10} C. 257_{10} D. 259_{10} E. None of these

QUESTION 2

Which of these values for `int x` would cause this expression to output "TRUE"?

```
if (x) cout << "TRUE";
```

- A. -1 B. 0 C. 1 D. Both A and C E. A, B, and C

QUESTION 3

If `int x` has the value 10 and `int y` has the value 17, what is the value of this expression?

```
(x<<2)^(y>>1)
```

- A. 32 B. 36 C. 40 D. 19 E. None of these

QUESTION 4

What is returned by the function to the right when it is passed this matrix?

1	3	2	3
2	3	1	3
1	3	2	3

A.

0
0
0

B.

0	0	0
0	0	0

C.

0	0	0
---	---	---

D.

0	0
0	0
0	0

- E. None of these

```
apmatrix<int> process(
    apmatrix<int> M) {
    int r = M[1][2],
        c = M[2][1];

    apmatrix<int> N(r,c,0);

    return N;
}
```

QUESTION 5

Which of these declares `discs` to hold 10 items of type `classical`?

- A. `apvector<classical> discs[10];`
- B. `apvector<classical> discs(10);`
- C. `apvector discs(classical, 10);`
- D. `apvector[10] discs<classical>;`
- E. A, B, and C

```
struct classical {
    apstring performer;
    apstring composer;
    apstring type;
    apstring key;
    int number;
};
```

QUESTION 6

An `apstring` has as data two `ints` and one `char *`. If `ints` and pointers are as big as 4 characters, what is `sizeof(classical)`?

- A. 5
- B. 52
- C. 34
- D. 16
- E. None of these

QUESTION 7

Which of these declares `c` to be of type `classical` and initializes it to Beethoven's *Piano Sonata No. 32 in C Minor* performed by Kovacevich?

- A. `classical c(Kovacevich, Beethoven, Piano Sonata, C Minor, 32);`
- B. `classical c("Kovacevich", "Beethoven", "Piano Sonata", "C Minor", "32");`
- C. `classical c;`
`c.performer = "Kovacevich";`
`c.composer = "Beethoven";`
`c.type = "Piano Sonata";`
`c.key = "C Minor";`
`c.number = "32";`
- D. `classical c;`
`c.performer = "Kovacevich";`
`c.composer = "Beethoven";`
`c.type = "Piano Sonata";`
`c.key = "C Minor";`
`c.number = 32;`
- E. None of these

QUESTION 8

What is the best case running time of the sequential search algorithm? Choose the smallest correct answer.

- A. $O(n \log n)$
- B. $O(n)$
- C. $O(\log n)$
- D. $O(1)$
- E. None of these

QUESTION 9

Which of the following replaces **<*1>** in the code to the right to declare a data member `child` which is an array of four pointers to type `panel`?

- A. `panel * child[4];`
- B. `panel * [4] child;`
- C. `panel[4] * child;`
- D. `panel ** child;`
- E. None of these

For the remaining questions, assume **<*1>** has been filled in correctly.

QUESTION 10

If `p1` and `p2` are `panel`s, which of these sets `p2` to be the northwest panel of `p1`?

- A. `p1.add_child(&p2, NW);`
- B. `p2.add_child(&p1, NW);`
- C. `p1.add_child(p2, NW);`
- D. `p2.add_child(p1, NW);`
- E. None of these

QUESTION 11

If `panel p` has these panels added as children, what will be the value of `p.h` and `p.w` after running the method `p.compute_size()`?

NE: no child

NW: panel with height 10 and width 45

SE: panel with height 15 and width 30

SW: panel with height 20 and width 10

- A. `p.h = 20, p.w = 45`
- B. `p.h = 35, p.w = 40`
- C. `p.h = 30, p.w = 45`
- D. `p.h = 45, p.w = 85`
- E. None of these

```
// Represents a panel for part of
// a windowing system. Each panel
// can hold four smaller panels.
// Data members representing the
// contents of the panels are
// omitted.
```

panel.h:

```
enum direction {NE, NW, SE, SW};
```

```
class panel {
```

```
public:
```

```
    panel(int height, int width);
```

```
    void resize(int newh, int neww);
```

```
    void add_child(panel * kid,
                  direction d);
```

```
    void compute_size();
```

```
private:
```

```
    int h,w;
```

```
    <*1>
```

```
};
```

panel.cpp:

```
#include "panel.h"
```

```
panel::panel(int height,
             int width) {
```

```
    h=height; w=width;
```

```
    for(int i=0; i<4; ++i)
```

```
        child[i]=NULL;
```

```
}
```

```
void panel::resize(int newh,
                  int neww) {
```

```
    h=newh, w=neww;
```

```
}
```

```
void panel::add_child(
```

```
    panel * kid, direction d) {
```

```
    child[d]=kid;
```

```
}
```

```
void panel::compute_size() {
```

```
    int w1=(child[0]?child[0]->w:0)+
        (child[1]?child[1]->w:0);
```

```
    int w2=(child[2]?child[2]->w:0)+
        (child[3]?child[3]->w:0);
```

```
    int h1=(child[0]?child[0]->h:0)+
        (child[2]?child[2]->h:0);
```

```
    int h2=(child[1]?child[1]->h:0)+
        (child[3]?child[3]->h:0);
```

```
    int h=(h1>h2)?h1:h2;
```

```
    int w=(w1>w2)?w1:w2;
```

```
}
```

QUESTION 12 What is returned by <code>mystery(15,2)</code>? A. 2 B. 15 C. 45 D. 30 E. None of these	<pre>int mystery(int x, int y) { if (y==0) return 0; int temp = y; x*=2; y/=2; if (temp%2==0) return mystery(x,y); else return x/2 + mystery(x,y); }</pre>
QUESTION 13 What is returned by <code>mystery(8,7)</code>? A. 7 B. 8 C. 27 D. 56 E. None of these	
QUESTION 14 What will be the value of <code>d</code> after executing the code to the right on the input below? 1.56e3 A. 1560 B. 1.56 C. .00156 D. 1.563 E. None of these	<pre>double d; cin >> d;</pre>
QUESTION 15 What replaces <code><*1></code> in the code to the right so that <code>apstring s</code> is output with all of its lower case letters replaced by upper case letters and with the other characters output unchanged? A. <code>s[i]+'A' - 'a'</code> B. <code>char(s[i]+'A' - 'a')</code> C. <code>char(toupper(s[i]))</code> D. <code>isupper(s[i])</code> E. More than one of these	<pre>void output(apstring s) { for (int i=0; i<s.length(); ++i) { cout << <*1>; } }</pre>
QUESTION 16 What is the maximum number of times the output statement will run in the code to the right? A. 1 B. 3 C. 5 D. 7 E. None of these	<pre>int i; cin >> i; if (i>100) i=100; do { cout << i; } while((i/=2)>0);</pre>

QUESTION 17

What is the return value of f(132)?

- A. 1 B. 2 C. 3
D. 4 E. None of these

```
int f(long x) {
    int y,z;
    if (x<0) x*=-1;
    while(x>10) {
        y=0, z=1;
        while(x) {
            y+=z*(x%10);
            z*=-1;
            x/=10;
        }
        x=y;
    }
    return x;
}
```

QUESTION 18

What is the return value of f(12344321)?

- A. 0 B. 1 C. 2
D. 3 E. None of these

QUESTION 19

What is returned by extract("12345")?

- A. 54321 B. 12345
C. "edcba" D. "abcde"
E. None of these

```
int extract(apstring s) {
    int x=0;
    for (int i=0;i<s.length();++i)
        if (s[i]>='0' && s[i]<='9')
            x=10*x+s[i]-'0';
    return x;
}
```

QUESTION 20

What is returned by extract("12ij23sn7")?

- A. 73221 B. 700320021
C. 12237 D. 120023007
E. None of these

QUESTION 21

Which statement below is represented by this truth table?

a	b	c	d
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

- A. $d = a || (b \& \& c)$ B. $d = (a \& \& b) || c$ C. $d = (a || b) \& \& c$
D. $d = (a \& \& c) || b$ E. None of these

QUESTION 22

Which of these replaces **<*1>** in the code to the right to put job j into the queue in array q given by the method of the class job named get_priority()?

- A. q[get_priority(j)].enqueue(j);
- B. enqueue(q[get_priority(j)],j);
- C. q[j.get_priority()].enqueue(j);
- D. enqueue(q, j, get_priority())
- E. None of these

For the remaining questions assume **<*1>** is filled in correctly.

QUESTION 23

For processor p declared in the main() function, which of these statements would be legal?

- A. job j=p.q[8].front() B. p.clock=0;
- C. p.timestep+=100; D. p.run();
- E. None of these

QUESTION 24

Which of these describes the behavior of the run() method?

- A. Jobs are run in the order that they arrive, regardless of their priority.
- B. Jobs are run in the reverse of the order that they arrive, regardless of their priority.
- C. Jobs are run in order of priority. Each job finishes running before the next job begins.
- D. Jobs are run in order of priority. Jobs of the same priority take turns running, with the first turn being taken in order of arrival.
- E. None of these

processor.h:

```
// The processor class is used in
// a simulation program. The job
// class (not shown) represents a
// computing job
```

```
class processor {
public:
    processor(int t);
    void add_job(job j);
    void run(int seconds);
private:
    apqueue<job> q[10];
    int clock;
    int timestep;
};
```

processor.cpp:

```
int max(int x, int y, int z) {
// returns max of x,y,z
}
```

```
processor::processor(int t) {
    timestep=t;
    clock=0;
}
```

```
void processor::add_job(job j) {
    j.set_time(clock);
    <*1>
}
```

```
void processor::run(int seconds) {
    int i=0;
    while(seconds>0) {
        if (i>=10) {
            clock+=seconds; seconds=0;
        }
        else if (q[i].isEmpty())
            ++i;
        else {
            job j;
            q[i].dequeue(j);
            int step = max(seconds,
                timestep, j.time_left());
            j.run(step);
            if (j.time_left())
                q[i].enqueue(j);
            else {
                j.summary();
            }
            seconds-=step;
            clock+=step;
        }
    }
}
```

QUESTION 25 What is output by the code to the right? A. 2 3 B. 3 3 C. 4 4 D. 3 4 E. None of these	<pre>int x=0, y=1; int f(int &y) { ++x; ++y; return x+y; } int main() { int x=2; int y = f(x); cout << x << ' ' << y; }</pre>
QUESTION 26 What is output by output("hi")? A. hihi B. hi02 C. hi D. 0202 E. None of these	<pre>void output(apstring s) { for (int i=0;i<s.length()-2;++i) s+= s[i] += s[i+2]-s[i+1]; cout << s; }</pre>
QUESTION 27 What is output by output("hello")? A. helloeo B. hello703 C. hello D. eo0loeo E. None of these	
QUESTION 28 What is output by the code to the right on this input? 23 43 12 0 A. CDBA B. CDEFBCDEFABCDEF C. FFFA D. CDEEBCDEABCDE E. None of these	<pre>int i; while (cin >> i) { switch(i/10) { case 0: cout << 'A'; case 1: cout << 'B'; case 2: cout << 'C'; case 3: cout << 'D'; case 4: cout << 'E'; default: cout << 'F'; } }</pre>
QUESTION 29 What is output by the code to the right on this input? -23 -43 -12 -0 A. CDBA B. CDEFBCDEFABCDEF C. FFFA D. CDEEBCDEABCDE E. A, B, and C	

QUESTION 30

Which of these replaces **<*1>** in the code to the right to declare a variable named `infile` for inputting the text file given by `s`?

- A. `infile(s);`
- B. `ifstream s;`
- C. `ifstream infile(s);`
- D. `infile = new(s);`
- E. None of these

```
apstring s;
cout << "Enter File:";
cin >> s;
```

<*1>

QUESTION 31

Which of these will create a stack with the values 3.75, 4.2, 5.5 and 6.4?

- A. `apstack<double> s;`
`s.push(3.75,4.2,`
`5.5,6.4);`
- B. `apstack<double> s;`
`s.add(3.75,4.2,`
`5.5,6.4);`
- C. `apstack<double> s;`
`s.push(3.75);`
`s.push(4.2);`
`s.push(5.5);`
`s.push(6.4);`
- D. `apstack<double> s;`
`s.add(3.75);`
`s.add(4.2);`
`s.add(5.5);`
`s.add(6.4);`
- E. More than one of these

```
void f(apstack<double> s) {
    double d=0;
    int count=0;
    while (!s.isEmpty()) {
        d+=s.top()*(++count);
        s.pop();
    }
    cout << setw(4) << d;
}
```

QUESTION 32

What will be output by a call to `f()` using the stack from the previous question?

- A. 54.25
- B. 34.4
- C. 19.85
- D. 19.9
- E. None of these

QUESTION 33

Which of the following replaces **<*1>** in the code to the right so that `find` correctly implements a binary search?

- A. `if (r<=s)`
- B. `if (r==s)`
- C. `if (r<s)`
- D. `if (r>s)`
- E. None of these

```
bool find(int A[], int r, int s,
          int item) {
    <*1>
    return false;
    int mid = (r+s)/2;
    if (A[mid]==item) return true;
    else if (A[mid]<item)
        return find(A,mid+1,s,item);
    else
        return find(A,r,mid-1,item);
}
```

QUESTION 34

When executing the function `quicksort(A, 0, 9)` on array A below, which of the following is *not* a possible representation of A after the first call to `partition()`?

11	19	23	17	8	28	10	7	4	3
----	----	----	----	---	----	----	---	---	---

A.	28	19	23	17	8	11	10	7	4	3
----	----	----	----	----	---	----	----	---	---	---

B.	8	7	4	3	10	11	17	19	23	28
----	---	---	---	---	----	----	----	----	----	----

C.	3	10	7	4	8	11	19	17	23	28
----	---	----	---	---	---	----	----	----	----	----

D.	3	8	4	7	10	11	28	23	19	27
----	---	---	---	---	----	----	----	----	----	----

E. More than one of these

```
int partition(int A[], int r,
              int s) {
    // splits A into elements
    // smaller and larger than A[r]
}

void quicksort(int A[], int r,
               int s) {
    if (r >= s) return;
    int mid = partition(A, r, s);
    quicksort(A, r, mid - 1);
    quicksort(A, mid + 1, s);
}
```

QUESTION 35

When executing the function `quicksort(A, 0, 9)` on array A below, which of the following is *not* a possible representation of A after the first recursive call to `quicksort()` finishes?

11	19	23	17	8	28	10	7	4	3
----	----	----	----	---	----	----	---	---	---

A.	3	4	7	8	10	17	23	19	11	28
----	---	---	---	---	----	----	----	----	----	----

B.	3	4	7	8	10	11	17	19	23	28
----	---	---	---	---	----	----	----	----	----	----

C.	3	4	7	8	10	11	19	23	28	17
----	---	---	---	---	----	----	----	----	----	----

D.	3	4	7	8	10	11	23	19	17	28
----	---	---	---	---	----	----	----	----	----	----

E. More than one of these

QUESTION 36

Which of the following describes the expected running time of `quicksort()` on a random array A of n elements? Choose the smallest valid answer.

- A. $O(n \log n)$ B. $O(n)$ C. $O(\log n)$
 D. $O(n^2)$ E. None of these

QUESTION 37

Which of these declares `iptr` to be a pointer to an array of 25 integers?

- A. `int *iptr = new int[25];` B. `int *iptr[25];` C. `int iptr = new[25];`
 D. `iptr = new int*[25];` E. More than one of these

QUESTION 38

How many '*'s are output by the code to the right if `n` is 8?

- A. 17 C. 24
 C. 8 D. 64
 E. None of these

```
for (int i=1; i<n; ++i)
  for (int j=1; j<n; j*=2)
    cout << '*';
```

QUESTION 39

What is the running time of the code to the right? Choose the smallest correct answer.

- A. $O(n \log n)$ B. $O(n)$
 C. $O(n^2)$ D. $O(1)$
 E. None of these

QUESTION 40

Which of the following function calls returns true?

- A. `test("");`
 B. `test("aaa");`
 C. `test("aaab");`
 D. `test("defgh");`
 E. None of these

```
bool test(apstring s) {
  int x=0;
  for (int i=0; i<s.length(); ++i) {
    switch(x) {
      case 0: x=s[i]-'a';
              break;
      case 1: ++x;
              break;
      case 2: return true;
      case 3: x=0;
              break;
      default: --x;
    }
  }
  return false;
}
```

AP QUICK REFERENCE

NOTE: The front and back of this page includes reference information for the AP classes. This is the same reference information that students use during the AP exam. You may use this reference sheet during the contest. You may detach this page from the test booklet.

Quick Reference for apstring

```
extern const int npos;    // used to indicate not a position in the string

class apstring

// public member functions

// constructors/destructor
apstring();               // construct empty string ""
apstring(const char * s); // construct from string literal
apstring(const apstring & str); // copy constructor
~apstring();              // destructor

// assignment
const apstring & operator= (const apstring & str); // assign str
const apstring & operator= (const char * s);       // assign s
const apstring & operator= (char ch);               // assign ch

// accessors
int length() const;           // number of chars
int find(const apstring & str) const; // index of first occurrence of str
int find(char ch) const;      // index of first occurrence of ch
apstring substr(int pos, int len) const; // substring of len chars, starting at pos
const char * c_str() const;    // explicit conversion to char *

// indexing
char operator[ ](int k) const; // range-checked indexing
char & operator[ ](int k);      // range-checked indexing

// modifiers
const apstring & operator+= (const apstring & str); // append str
const apstring & operator+= (char ch);              // append char

// The following free (non-member) functions operate on strings

// I/O functions
ostream & operator<< ( ostream & os, const apstring & str );
istream & operator>> ( istream & is, apstring & str );
istream & getline( istream & is, apstring & str );

// comparison operators
bool operator== ( const apstring & lhs, const apstring & rhs );
bool operator!= ( const apstring & lhs, const apstring & rhs );
bool operator<  ( const apstring & lhs, const apstring & rhs );
bool operator<= ( const apstring & lhs, const apstring & rhs );
bool operator>  ( const apstring & lhs, const apstring & rhs );
bool operator>= ( const apstring & lhs, const apstring & rhs );

// concatenation operator +
apstring operator+ ( const apstring & lhs, const apstring & rhs );
apstring operator+ ( char ch, const apstring & str );
apstring operator+ ( const apstring & str, char ch );
```

AP QUICK REFERENCE CONT.

Quick Reference for apvector and apmatrix

```
template <class itemType>
class apvector

// public member functions

// constructors/destructor
apvector(); // default constructor (size==0)
apvector(int size); // initial size of vector is size
apvector(int size, const itemType & fillValue); // all entries == fillValue
apvector(const apvector & vec); // copy constructor
~apvector(); // destructor

// assignment
const apvector & operator= (const apvector & vec);

// accessors
int length() const; // capacity of vector

// indexing
itemType & operator[ ](int index); // indexing with range checking
const itemType & operator[ ](int index) const; // indexing with range checking

// modifiers
void resize(int newSize); // change size dynamically; can result in losing values
```

```
template <class itemType>
class apmatrix

// public member functions

// constructors/destructor
apmatrix(); // default size is 0 x 0
apmatrix(int rows, int cols); // size is rows x cols
apmatrix(int rows, int cols, const itemType & fillValue); // all entries==fillValue
apmatrix(const apmatrix & mat); // copy constructor
~apmatrix( ); // destructor

// assignment
const apmatrix & operator = (const apmatrix & rhs);

// accessors
int numrows() const; // number of rows
int numcols() const; // number of columns

// indexing
const apvector<itemType> & operator[ ](int k) const; // range-checked indexing
apvector<itemType> & operator[ ](int k); // range-checked indexing

// modifiers
void resize(int newRows, int newCols); // resizes matrix to newRows x newCols
// (can result in losing values)
```

AP QUICK REFERENCE CONT.

Quick Reference for apstack and apqueue

```
template <class itemType>
class apstack

// public member functions

// constructors/destructor
apstack(); // construct empty stack
apstack(const apstack & s); // copy constructor
~apstack(); // destructor

// assignment
const apstack & operator= (const apstack & s);

// accessors
const itemType & top() const; // return top element (NO pop)
bool isEmpty() const; // return true if empty, else false
int length() const; // returns number of elements in stack

// modifiers
void push(const itemType & item); // push item onto top of stack
void pop(); // pop top element
void pop(itemType & item); // combines pop and top
void makeEmpty(); // make stack empty (no elements)

template <class itemType>
class apqueue

// public member functions

// constructors/destructor
apqueue(); // construct empty queue
apqueue(const apqueue & q); // copy constructor
~apqueue(); // destructor

// assignment
const apqueue & operator = (const apqueue & rhs);

// accessors
const itemType & front() const; // return front (no dequeue)
bool isEmpty() const; // return true if empty else false
int length() const; // return number of elements in queue

// modifiers
void enqueue(const itemType & item); // insert item (at rear)
void dequeue(); // remove first element
void dequeue(itemType & item); // combine front and dequeue
void makeEmpty(); // make queue empty
```

Computer Science Answer Key

UIL Regional 2003

1. D	11. C	21. D	31. C
2. D	12. D	22. C	32. E
3. A	13. C	23. E	33. D
4. A	14. A	24. D	34. A
5. C	15. C	25. D	35. A
6. B	16. D	26. C	36. A
7. D	17. E	27. E	37. A
8. D	18. A	28. B	38. B
9. A	19. B	29. C	39. A
10. A	20. C	30. E	40. E